МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОТЧЁТ

по лабораторной работе № 7

«Рекурсивные алгоритмы и динамическое программирование в задачах комбинаторной оптимизации»

Дисциплина: **Теория алгоритмов**

Направление подготовки: 38.03.05 «Бизнес-информатика»

Вариант 13

Выполнила: студентка группы БИ 2-1

Кугушева Арина Вячеславовна

Проверил: _____

2026

1. Цель работы и задание варианта

Цель работы: изучить динамическое программирование как способ устранения повторных вычислений в рекурсивных алгоритмах и экспериментально сопоставить три реализации одной задачи — наивную рекурсивную, рекурсивную с мемоизацией и итеративную; решить задачу дискретной оптимизации с восстановлением оптимального решения.

Задание (вариант 13). Задача А: число способов подняться по лестнице из n ступеней шагами по 1, 2 или 3 ступени; реализовать тремя способами с подсчётом вызовов, n = 10…27, определить предел наивной рекурсии (ориентир — 10 с). **Задача Б:** редакционное расстояние (расстояние Левенштейна) между двумя строками длиной 300 символов — решить методом ДП, восстановить последовательность операций, сверить с полным перебором на малых строках. **Дополнительное задание Г6:** сравнить собственную мемоизацию с декоратором `functools.lru_cache` и объяснить разницу во времени.

№	Задача А	Параметр n	Задача Б	Данные задачи Б	Доп.
13	Лестница, шаги 1, 2, 3	10…27	Редакционное расстояние	Строки длиной 300	Г6

Условия эксперимента. Время измерялось функцией `time.perf_counter()`. Для быстрых вариантов вызов повторяли серией и брали медиану по пяти сериям; для наивной рекурсии при n ≥ 26 — одиночный замер. Каждая реализация задачи А возвращает пару «значение, число вызовов»; у итеративной версии вместо числа вызовов считаются шаги цикла (вызов один). Строка A длиной 300 — случайный текст над русским алфавитом (27 букв, генератор с начальным значением 13); строка B получена из A тридцатью случайными правками (замены и «сдвиги» символа) при сохранении длины. Для сверки с полным перебором использовались 200 пар случайных строк длиной до 7 символов.

2. Рекуррентные соотношения

Задача А. Пусть f(n) — число способов подняться на n-ю ступень шагами по 1, 2 или 3 ступени. На n-ю ступень можно попасть с ступени n − 1 (шаг 1), n − 2 (шаг 2) или n − 3 (шаг 3), и эти группы способов не пересекаются, поэтому

$$f(n) = f(n-1) + f(n-2) + f(n-3),\ f(0) = f(1) = 1,\ f(2) = 2.$$

Слагаемые: f(n − 1) — способы, у которых последний шаг равен 1; f(n − 2) — последний шаг 2; f(n − 3) — последний шаг 3. Базовые значения: на нулевой ступени (старт) и на первой — единственный способ; на второй — два (1 + 1 или 2). Первые значения: 1, 1, 2, 4, 7, 13, 24, …

Число вызовов наивной рекурсии удовлетворяет соотношению T(n) = 1 + T(n − 1) + T(n − 2) + T(n − 3), T(0) = T(1) = T(2) = 1 (единица — сам вызов), и растёт как $\Theta(c^n)$, где c ≈ 1,8393 — действительный корень уравнения $x^3 = x^2 + x + 1$ (постоянная «трибоначчи»). **С мемоизацией** нетривиальных подзадач (k = 3…n) всего n − 2, каждая вычисляется один раз и порождает три вызова, поэтому число вызовов равно 1 + 3(n − 2) = 3n − 5 = Θ(n). **Итеративная версия** выполняет n − 2 шага цикла и хранит три последних значения: Θ(n) времени, O(1) памяти.

Задача Б. Пусть D[i][j] — минимальное число операций (вставка, удаление, замена одного символа, цена каждой 1), превращающих первые i символов строки A в первые j символов строки B. Тогда

$$D[i][0] = i,\ D[0][j] = j,$$

D[i][j] = D[i − 1][j − 1], если A[i] = B[j]; D[i][j] = 1 + min(D[i − 1][j], D[i][j − 1], D[i − 1][j − 1]) иначе.

Слагаемые: D[i − 1][j] + 1 — *удаление* символа A[i]; D[i][j − 1] + 1 — *вставка* символа B[j] в конец; D[i − 1][j − 1] + 1 — *замена* A[i] на B[j]; при A[i] = B[j] последние символы совпадают, и расстояние равно расстоянию между более короткими префиксами (это не хуже любой операции). Граничные значения: превратить пустую строку в j символов — j вставок, i символов в пустую — i удалений. Ответ — D[m][n]. Свойства динамического программирования выполнены: оптимальная подструктура (оптимальное преобразование префиксов содержит оптимальные преобразования более коротких префиксов) и перекрывающиеся подзадачи ((m + 1)(n + 1) различных пар (i, j) при экспоненциальном числе путей в рекурсии). Трудоёмкость таблицы $\Theta(mn)$ по времени и памяти (при m = n = 300 — 90 601 клетка); двухстрочная версия использует O(n) памяти, но не позволяет восстановить операции. Наивная рекурсия делает порядка $(3 + 2\sqrt{2})^n \approx 5{,}83^n$ вызовов.

Восстановление решения. Обратный проход от клетки (m, n) к (0, 0): если A[i] = B[j] и D[i][j] = D[i − 1][j − 1] — совпадение; иначе если D[i][j] = D[i − 1][j − 1] + 1 — замена; иначе если D[i][j] = D[i − 1][j] + 1 — удаление; иначе — вставка (приоритет при равенстве фиксирован, поэтому результат воспроизводим).

3. Дерево рекурсии и блок-схемы

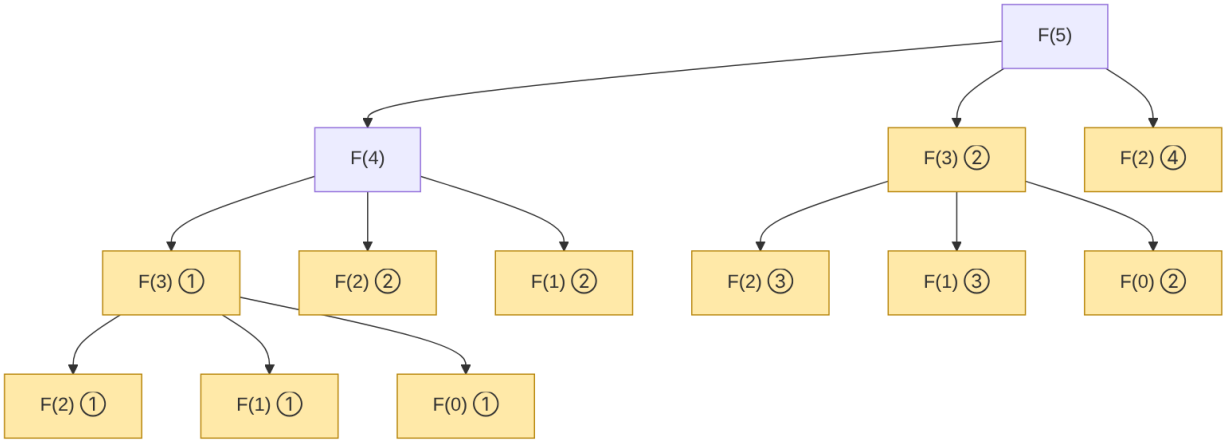


Рис. 1. Дерево рекурсии наивной версии для f(5): 13 вызовов; повторяющиеся подзадачи выделены цветом и пронумерованы (F(3) — 2 раза, F(2) — 4 раза, F(1) — 3 раза, F(0) — 2 раза)

На рис. 1 подзадача F(3) решается дважды, F(2) — четыре раза, F(1) — трижды, F(0) — дважды. При мемоизации каждая подзадача вычисляется один раз, и дерево вырождается в «позвоночник» F(5) → F(4) → F(3) с обращениями к кэшу вместо повторных поддеревьев.

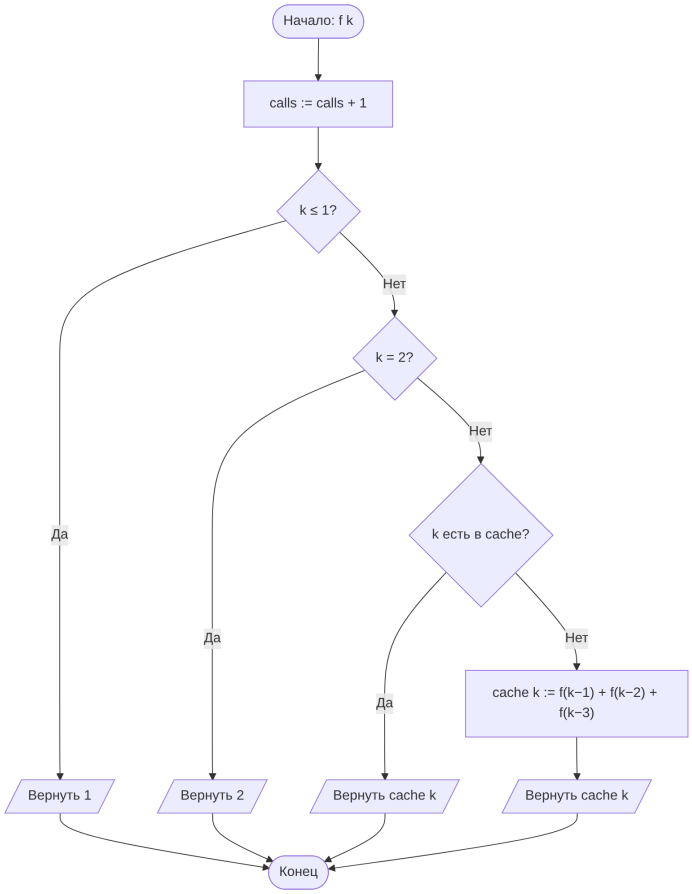


Рис. 2. Блок-схема мемоизированной функции задачи A

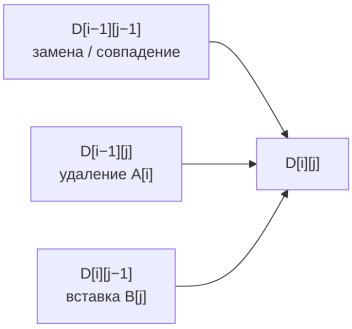


Рис. 3. Схема зависимости клетки таблицы D[i][j] от трёх соседних клеток

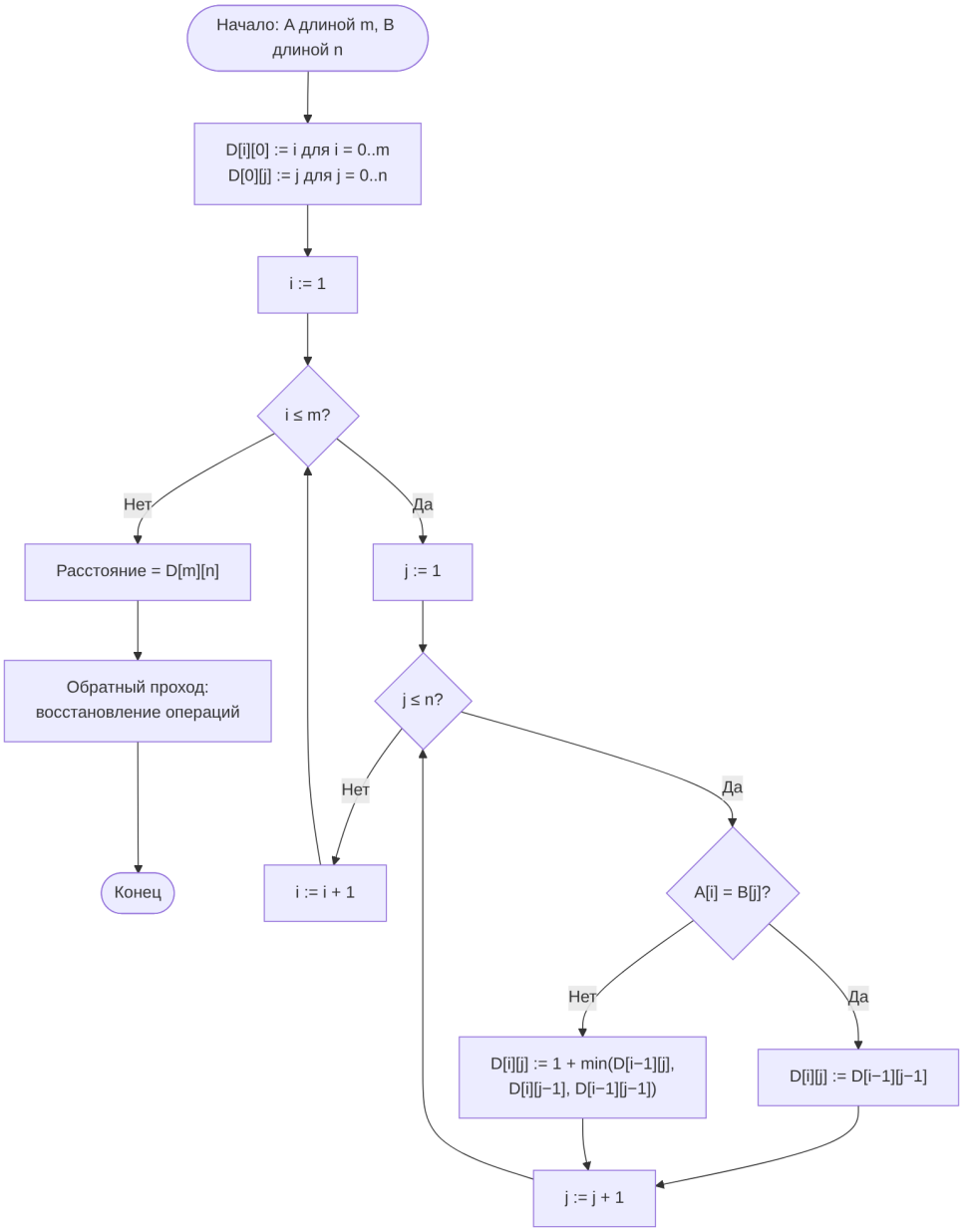


Рис. 4. Блок-схема заполнения таблицы ДП для редакционного расстояния

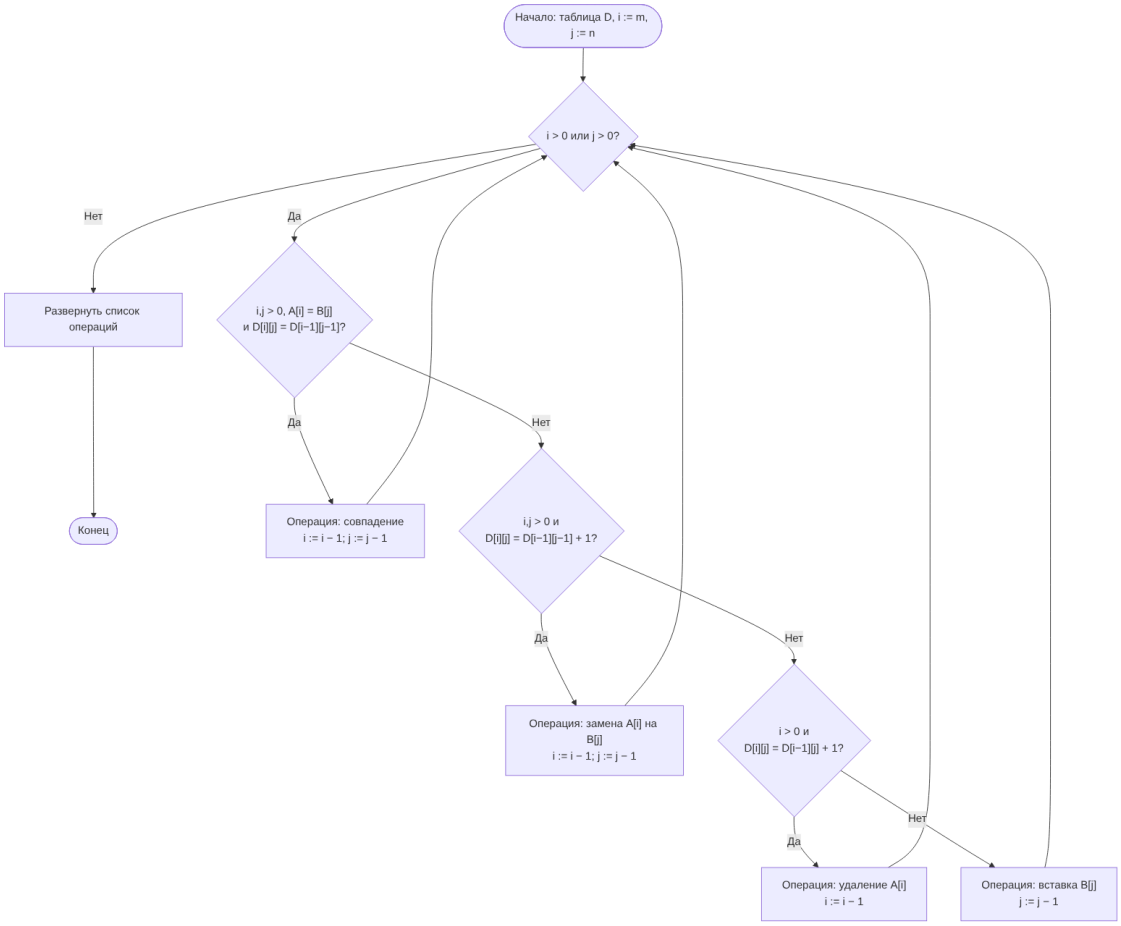


Рис. 5. Блок-схема восстановления последовательности операций (обратный проход)

4. Листинг программы

Файл `lr7_variant13.py` (самодостаточный модуль с подробными комментариями: три реализации задачи А, четыре реализации задачи Б, восстановление решения, мемоизации для сравнения, самопроверка, эксперименты, графики). Запуск: `python lr7_variant13.py`; требуется matplotlib.

```
"""
Лабораторная работа № 7. Вариант 13.
Дисциплина: Теория алгоритмов.
Выполнила: Кугушева Арина Вячеславовна, группа БИ 2-1.

Тема: рекурсивные алгоритмы и динамическое программирование (ДП) в задачах
комбинаторной оптимизации.

ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА 13 (табл. 1 задания)
    задача А (три реализации) .. число способов подняться по лестнице из n
                                 ступеней шагами по 1, 2 или 3 ступени;
                                 параметр n = 10 ... 27;
    задача Б (ДП) ............... редакционное расстояние (Левенштейна) между
                                 двумя строками длиной 300 символов;
    дополнительное задание Г6 .. сравнить собственную мемоизацию с декоратором
                                 functools.lru_cache и объяснить разницу во времени.

ЗАДАЧА А. Рекуррентное соотношение:
    f(0) = f(1) = 1,  f(2) = 2,  f(n) = f(n-1) + f(n-2) + f(n-3)  при n >= 3.
    (На последней ступени оказываются, сделав шаг в 1, 2 или 3 ступени с
    ступеней n-1, n-2, n-3 соответственно.) Три реализации: наивная рекурсия,
    рекурсия с мемоизацией, итеративное решение; все считают число вызовов.

ЗАДАЧА Б. Редакционное расстояние D[i][j] между первыми i символами строки A
и первыми j символами строки B (операции: вставка, удаление, замена, цена 1):
    D[i][0] = i,  D[0][j] = j,
    D[i][j] = D[i-1][j-1],                                  если A[i] = B[j],
    D[i][j] = 1 + min(D[i-1][j], D[i][j-1], D[i-1][j-1]),   иначе.
Программа не только считает расстояние, но и ВОССТАНАВЛИВАЕТ оптимальную
последовательность операций обратным проходом по таблице.

ЧТО ДЕЛАЕТ ПРОГРАММА (по порядку)
    0. Самопроверка: ДП сверяется с полным перебором на 200 парах малых строк,
       восстановленная последовательность операций применяется к строке A и
       должна дать строку B.
    1. Дерево рекурсии для f(5) и таблица ДП для малого примера.
    2. Задача А: число вызовов и время трёх реализаций при n = 10 ... 27,
       оценка основания экспоненты, поиск n, при котором наивная рекурсия
       перестаёт укладываться в 10 секунд.
    3. Задача Б: оптимальное решение для строк длиной 300 (с расшифровкой);
       наивная рекурсия, мемоизация и таблица на малых длинах; рост времени ДП
       с длиной строк (должен быть квадратичным).
    4. Г6: собственная мемоизация и lru_cache - время и число вызовов.

ЗАПУСК
    python lr7_variant13.py             (полный прогон, около 1-1,5 минуты)
    python lr7_variant13.py --json F    (дополнительно сохранить результаты в F)
    python lr7_variant13.py --show      (дополнительно показать графики)

Требуется: Python 3.10+, библиотека matplotlib (pip install matplotlib).
Результат: таблицы в консоли и графики lr7_plot_stairs.png,
lr7_plot_edit.png, lr7_plot_cache.png в текущей папке.
"""

import functools
import json
import math
import platform
import random
import statistics
import sys
import time

import matplotlib

if "--show" not in sys.argv:
    matplotlib.use("Agg")          # режим без окна: графики только сохраняются в файлы
import matplotlib.pyplot as plt

sys.setrecursionlimit(20_000)      # мемоизированная рекурсия для строк длиной 300 уходит на ~600 уровней

# ---------------------------------------------------------------------------
# 1. ПАРАМЕТРЫ ЭКСПЕРИМЕНТА
# ---------------------------------------------------------------------------
STAIRS_N = list(range(10, 28))      # параметр n задачи А: 10 ... 27 (вариант 13)
```

```
LIMIT_SECONDS = 10.0                # «разумное время» для наивной рекурсии (ориентир из задания)
LIMIT_SEARCH_START = 28             # с этого n ищем предел наивной рекурсии
LIMIT_SEARCH_MAX = 36               # выше не идём в любом случае
STRING_LEN = 300                    # длина строк задачи Б (вариант 13)
MUTATIONS = 30                      # число случайных правок, превращающих A в B
ALPHABET = "абвгдежзиклмнопрстуфхцчшэюя"   # алфавит для генерации строк
NAIVE_EDIT_LENGTHS = list(range(2, 9))      # длины строк для наивного перебора в задаче Б
EDIT_LENGTHS = [50, 100, 150, 200, 300, 400]   # длины для замеров роста времени ДП
CACHE_BIG_N = [100, 300, 900]       # большие n для сравнения мемоизаций в задаче А
CACHE_EDIT_LENGTHS = [100, 200, 300]   # длины для сравнения мемоизаций в задаче Б
SEED = 13                           # начальное значение генератора случайных чисел


# ---------------------------------------------------------------------------
# 2. ЗАДАЧА А: ЧИСЛО СПОСОБОВ ПОДНЯТЬСЯ ПО ЛЕСТНИЦЕ (ШАГИ 1, 2, 3)
# ---------------------------------------------------------------------------
def stairs_naive(n):
    """
    НАИВНАЯ рекурсия по определению f(n) = f(n-1) + f(n-2) + f(n-3).
    Одни и те же подзадачи решаются многократно, число вызовов растёт
    экспоненциально: T(n) = 1 + T(n-1) + T(n-2) + T(n-3), T(0) = T(1) = T(2) = 1,
    то есть θ(c^n), где c ≈ 1,8393 - корень уравнения x^3 = x^2 + x + 1.
    Возвращает пару (значение f(n), число вызовов функции).
    """
    calls = [0]                                     # счётчик вызовов (список - чтобы менять из вложенной функции)

    def f(k):
        calls[0] += 1                               # каждый вызов, включая базовые, учитывается
        if k <= 1:                                  # базовый случай: f(0) = f(1) = 1
            return 1
        if k == 2:                                  # базовый случай: f(2) = 2
            return 2
        return f(k - 1) + f(k - 2) + f(k - 3)       # шаг рекурсии

    return f(n), calls[0]


def stairs_memo(n):
    """
    РЕКУРСИЯ С МЕМОИЗАЦИЕЙ (нисходящее ДП): результат каждой подзадачи один раз
    записывается в словарь cache и затем берётся оттуда.
    Нетривиальных подзадач (k = 3 ... n) всего n - 2, каждая вычисляется один
    раз и порождает три вызова, поэтому число вызовов равно 1 + 3(n - 2) = 3n - 5
    при n >= 3, то есть θ(n); память θ(n) на кэш и стек рекурсии глубиной n.
    Возвращает пару (значение, число вызовов).
    """
    cache = {}                                      # кэш: номер ступени -> число способов
    calls = [0]

    def f(k):
        calls[0] += 1
        if k <= 1:
            return 1
        if k == 2:
            return 2
        if k in cache:                              # подзадача уже решена - берём из кэша
            return cache[k]
        cache[k] = f(k - 1) + f(k - 2) + f(k - 3)   # решаем один раз и запоминаем
        return cache[k]

    return f(n), calls[0]


def stairs_iter(n):
    """
    ИТЕРАТИВНОЕ решение (восходящее ДП): значения f(0), f(1), ... вычисляются по
    порядку, хранятся только три последних. θ(n) времени, O(1) памяти,
    рекурсии нет - нет и ограничения на глубину стека.
    Возвращает пару (значение, число «вызовов»). Вызов тут один (функции
    не вызывают друг друга), а число шагов цикла - n - 2, поэтому в счётчик
    записывается именно число шагов цикла (+1 за сам вызов), чтобы сравнивать
    объём работы с другими версиями.
    """
    if n <= 1:
        return 1, 1
    if n == 2:
        return 2, 1
    a, b, c = 1, 1, 2                               # f(0), f(1), f(2)
    steps = 1                                       # один вызов самой функции
    for _ in range(3, n + 1):
        a, b, c = b, c, a + b + c                   # сдвиг окна из трёх значений
        steps += 1
    return c, steps
def stairs_calls_formula(n):
    """Число вызовов наивной рекурсии по формуле T(n) = 1 + T(n-1) + T(n-2) + T(n-3)."""
```

```
    t = [1, 1, 1]                                   # T(0), T(1), T(2)
    for k in range(3, n + 1):
        t.append(1 + t[k - 1] + t[k - 2] + t[k - 3])
    return t[n]


def recursion_tree_stairs(n=5):
    """
    Дерево рекурсии наивной версии для f(n) в формате Mermaid (для отчёта).
    Повторяющиеся подзадачи помечаются номером вхождения: F(3) □, F(3) □ ...
    Возвращает строку с кодом диаграммы и словарь «подзадача -> число вхождений».
    """
    lines, counts, seen = ["flowchart TD"], {}, {}
    uid = [0]

    def build(k):
        uid[0] += 1
        node = f"N{uid[0]}"
        counts[k] = counts.get(k, 0) + 1
        seen[k] = seen.get(k, 0) + 1
        label = f"F({k})"
        lines.append(f'    {node}["{label}"]')       # подпись уточняется после обхода
        if k > 2:
            for d in (1, 2, 3):                     # три дочерних вызова
                child = build(k - d)
                lines.append(f"    {node} --> {child}")
        return node

    build(n)
    # проставляем номера вхождений для подзадач, которые повторяются
    order, out = {}, []
    for line in lines:
        if line.strip().startswith("N") and '["F(' in line:
            k = int(line.split("F(")[1].split(")")[0])
            if counts[k] > 1:
                order[k] = order.get(k, 0) + 1
                mark = "□□□□□□□□□"[order[k] - 1]
                line = line.replace(f'F({k})"', f'F({k}) {mark}"')
        out.append(line)
    return "\n".join(out), counts


# ---------------------------------------------------------------------------
# 3. ЗАДАЧА Б: РЕДАКЦИОННОЕ РАССТОЯНИЕ
# ---------------------------------------------------------------------------
def edit_naive(a, b):
    """
    ПОЛНЫЙ ПЕРЕБОР (наивная рекурсия по определению). Годится только для очень
    коротких строк: число вызовов растёт примерно как (3 + 2·√2)^n ≈ 5,83^n.
    Возвращает пару (расстояние, число вызовов).
    """
    calls = [0]

    def d(i, j):                                    # расстояние между a[:i] и b[:j]
        calls[0] += 1
        if i == 0:                                  # A пуста: нужно вставить j символов
            return j
        if j == 0:                                  # B пуста: нужно удалить i символов
            return i
        if a[i - 1] == b[j - 1]:                    # последние символы совпали - ничего не делаем
            return d(i - 1, j - 1)
        return 1 + min(d(i - 1, j),                 # удаление a[i-1]
                       d(i, j - 1),                 # вставка b[j-1]
                       d(i - 1, j - 1))             # замена a[i-1] на b[j-1]

    return d(len(a), len(b)), calls[0]


def edit_memo(a, b):
    """
    Рекурсия с МЕМОИЗАЦИЕЙ: словарь cache хранит D[i][j]. Различных подзадач
    (m + 1)(n + 1), каждая считается один раз: Θ(mn) времени и памяти.
    Возвращает пару (расстояние, число вызовов).
    """
    cache = {}
    calls = [0]

    def d(i, j):
        calls[0] += 1
        if i == 0:
            return j
        if j == 0:
            return i
        key = (i, j)
        if key in cache:                            # подзадача уже решена
```

```
            return cache[key]
        if a[i - 1] == b[j - 1]:
            res = d(i - 1, j - 1)
        else:
            res = 1 + min(d(i - 1, j), d(i, j - 1), d(i - 1, j - 1))
        cache[key] = res
        return res

    return d(len(a), len(b)), calls[0]


def edit_table(a, b):
    """
    ВОСХОДЯЩЕЕ ДП: заполняется вся таблица D размером (m + 1) x (n + 1) строка
    за строкой слева направо; рекурсии нет. Θ(mn) времени и памяти.
    Возвращает таблицу D (список списков); расстояние лежит в D[m][n].
    """
    m, n = len(a), len(b)
    D = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):                          # первый столбец: D[i][0] = i
        D[i][0] = i
    for j in range(n + 1):                          # первая строка: D[0][j] = j
        D[0][j] = j
    for i in range(1, m + 1):
        row, prev = D[i], D[i - 1]                  # локальные ссылки на строки - чуть быстрее
        ai = a[i - 1]
        for j in range(1, n + 1):
            if ai == b[j - 1]:
                row[j] = prev[j - 1]                # символы совпали
            else:
                row[j] = 1 + min(prev[j],           # удаление
                                 row[j - 1],        # вставка
                                 prev[j - 1])       # замена
    return D


def edit_two_rows(a, b):
    """
    ДП с ОПТИМИЗАЦИЕЙ ПАМЯТИ: в каждый момент нужны только предыдущая и текущая
    строки таблицы, поэтому память O(n) вместо O(mn). Платой за это служит
    невозможность восстановить последовательность операций по таблице.
    Возвращает расстояние.
    """
    n = len(b)
    prev = list(range(n + 1))                       # строка D[0][*]
    for i in range(1, len(a) + 1):
        cur = [i] + [0] * n                         # D[i][0] = i
        ai = a[i - 1]
        for j in range(1, n + 1):
            if ai == b[j - 1]:
                cur[j] = prev[j - 1]
            else:
                cur[j] = 1 + min(prev[j], cur[j - 1], prev[j - 1])
        prev = cur
    return prev[n]


def edit_script(a, b, D):
    """
    ВОССТАНОВЛЕНИЕ решения обратным проходом по таблице D: от клетки (m, n) к
    (0, 0). В каждой клетке выясняем, из какой соседней клетки получено её
    значение. Приоритет при равенстве: совпадение/замена, затем удаление,
    затем вставка (так результат воспроизводим).

    Возвращает список операций от начала строки к концу. Каждая операция -
    кортеж (название, i, j, символ_A, символ_B), где i и j - позиции (с 1)
    в строках A и B; названия: "совпадение", "замена", "удаление", "вставка".
    """
    ops = []
    i, j = len(a), len(b)
    while i > 0 or j > 0:
        if i > 0 and j > 0 and a[i - 1] == b[j - 1] and D[i][j] == D[i - 1][j - 1]:
            ops.append(("совпадение", i, j, a[i - 1], b[j - 1]))
            i, j = i - 1, j - 1
        elif i > 0 and j > 0 and D[i][j] == D[i - 1][j - 1] + 1:
            ops.append(("замена", i, j, a[i - 1], b[j - 1]))
            i, j = i - 1, j - 1
        elif i > 0 and D[i][j] == D[i - 1][j] + 1:
            ops.append(("удаление", i, j, a[i - 1], ""))
            i -= 1
        else:                                       # остаётся вставка: D[i][j] = D[i][j-1] + 1
            ops.append(("вставка", i, j, "", b[j - 1]))
            j -= 1
    ops.reverse()                                   # шли с конца - разворачиваем
    return ops
```

```
def apply_script(a, ops):
    """Применяет последовательность операций к строке A и возвращает полученную строку."""
    result = []
    for name, i, j, ca, cb in ops:
        if name in ("совпадение", "замена", "вставка"):
            result.append(cb)                       # символ B попадает в результат
        # при удалении символ A просто пропускается
    return "".join(result)


def random_text(rng, n):
    """Случайная строка длиной n над русским алфавитом."""
    return "".join(rng.choice(ALPHABET) for _ in range(n))


def mutate(rng, a, k):
    """
    Строка B из A: выполняется k случайных правок, причём длина сохраняется
    (поэтому обе строки имеют одинаковую длину, 300). Правка - это либо замена
    случайного символа на случайный, либо «сдвиг»: удаление символа в одной
    позиции и вставка случайного символа в другой.
    """
    chars = list(a)
    for _ in range(k):
        if rng.random() < 0.5:                      # замена символа
            chars[rng.randrange(len(chars))] = rng.choice(ALPHABET)
        else:                                       # удаление + вставка в другом месте
            del chars[rng.randrange(len(chars))]
            chars.insert(rng.randrange(len(chars) + 1), rng.choice(ALPHABET))
    return "".join(chars)


# ---------------------------------------------------------------------------
# 4. ГЕНЕРИЧЕСКИЕ МЕМОИЗАЦИИ ДЛЯ СРАВНЕНИЯ (Г6)
# ---------------------------------------------------------------------------
def my_memoize(func):
    """
    СОБСТВЕННЫЙ декоратор мемоизации (чистый Python): словарь cache и обёртка.
    Ключом служит кортеж аргументов. Статистика calls / misses нужна для отчёта.
    """
    cache = {}

    @functools.wraps(func)
    def wrapper(*args):
        wrapper.calls += 1                          # все обращения к функции
        if args in cache:                           # попадание в кэш
            return cache[args]
        wrapper.misses += 1                         # реальное вычисление
        result = func(*args)
        cache[args] = result
        return result

    wrapper.calls = 0
    wrapper.misses = 0
    return wrapper


def make_stairs_decorated(kind):
    """
    Создаёт НОВУЮ (с пустым кэшем) мемоизированную функцию f(n) задачи A.
    kind: "own" - собственный декоратор my_memoize, "lru" - functools.lru_cache.
    """
    def f(k):
        if k <= 1:
            return 1
        if k == 2:
            return 2
        return g(k - 1) + g(k - 2) + g(k - 3)       # рекурсия идёт через мемоизированную g

    g = my_memoize(f) if kind == "own" else functools.lru_cache(maxsize=None)(f)
    return g


def make_edit_decorated(kind, a, b):
    """То же для редакционного расстояния: возвращает мемоизированную d(i, j)."""
    def d(i, j):
        if i == 0:
            return j
        if j == 0:
            return i
        if a[i - 1] == b[j - 1]:
            return g(i - 1, j - 1)
        return 1 + min(g(i - 1, j), g(i, j - 1), g(i - 1, j - 1))

    g = my_memoize(d) if kind == "own" else functools.lru_cache(maxsize=None)(d)
```

```
    return g


def cache_calls(g, kind):
    """(всего обращений, реальных вычислений) для мемоизированной функции g."""
    if kind == "own":
        return g.calls, g.misses
    info = g.cache_info()
    return info.hits + info.misses, info.misses


# ---------------------------------------------------------------------------
# 5. ИЗМЕРЕНИЯ
# ---------------------------------------------------------------------------
def timed(func):
    """Время выполнения func() в секундах (и её результат)."""
    t0 = time.perf_counter()
    res = func()
    return time.perf_counter() - t0, res


def avg_time(func, min_total=0.05, max_reps=20_000):
    """
    Среднее время одного вызова func() в секундах для БЫСТРЫХ функций: вызовы
    повторяются, пока суммарное время не превысит min_total. Возвращает медиану
    по пяти таким сериям.
    """
    t, _ = timed(func)                              # пробный вызов для оценки времени
    reps = max(1, min(max_reps, int(min_total / max(t, 1e-7))))
    series = []
    for _ in range(5):
        t0 = time.perf_counter()
        for _ in range(reps):
            func()
        series.append((time.perf_counter() - t0) / reps)
    return statistics.median(series)


def median_run(maker, reps):
    """
    Медиана времени для функций, которым нужен СВЕЖИЙ кэш: maker() создаёт
    функцию заново (создание в замер не входит), затем замеряется один вызов.
    maker должна возвращать пару (функция без аргументов, объект_для_счётчика).
    """
    times = []
    for _ in range(reps):
        call, _ = maker()
        t, _ = timed(call)
        times.append(t)
    return statistics.median(times)


def experiment_stairs():
    """Задача A: число вызовов и время трёх реализаций при n = 10 ... 27."""
    rows = []
    for n in STAIRS_N:
        v1, c1 = stairs_naive(n)
        v2, c2 = stairs_memo(n)
        v3, c3 = stairs_iter(n)
        assert v1 == v2 == v3, "реализации дали разные значения"
        assert c1 == stairs_calls_formula(n), "число вызовов не совпало с формулой T(n)"
        assert c2 == 3 * n - 5, "число вызовов мемоизации не совпало с 3n - 5"
        if n <= 22:                                 # быстрые случаи измеряем усреднением
            t1 = avg_time(lambda: stairs_naive(n), 0.2, 50)
        else:                                       # крупные - единичным замером (медиана из 3 для n <= 25)
            reps = 3 if n <= 25 else 1
            t1 = statistics.median(timed(lambda: stairs_naive(n))[0] for _ in range(reps))
        rows.append({
            "n": n, "value": v1,
            "calls_naive": c1, "calls_memo": c2, "calls_iter": c3,
            "t_naive": t1,
            "t_memo": avg_time(lambda: stairs_memo(n)),
            "t_iter": avg_time(lambda: stairs_iter(n)),
        })
    return rows


def experiment_limit(last_n, last_time):
    """
    Поиск n, при котором наивная рекурсия перестаёт укладываться в LIMIT_SECONDS.
    Запускаем наивную версию для n = LIMIT_SEARCH_START, +1, ... до первого
    превышения порога (но не выше LIMIT_SEARCH_MAX).
    """
    out = []
    for n in range(LIMIT_SEARCH_START, LIMIT_SEARCH_MAX + 1):
```

```
        t, (v, c) = timed(lambda: stairs_naive(n))
        out.append({"n": n, "t": t, "calls": c, "value": v})
        print(f"    n = {n}: {t:.2f} c, {c} вызовов", flush=True)
        if t > LIMIT_SECONDS:
            break
    return out


def fit_base(rows):
    """Основание экспоненты: среднее T(n+1)/T(n) и оценка по наклону log T от n."""
    ratios = [rows[i + 1]["calls_naive"] / rows[i]["calls_naive"] for i in range(len(rows) - 1)]
    xs = [r["n"] for r in rows]
    ys = [math.log(r["calls_naive"]) for r in rows]
    mx, my = statistics.mean(xs), statistics.mean(ys)
    slope = sum((x - mx) * (y - my) for x, y in zip(xs, ys)) / sum((x - mx) ** 2 for x in xs)
    return {"last_ratio": ratios[-1], "mean_ratio": statistics.mean(ratios),
            "slope_base": math.exp(slope)}


def experiment_edit_main():
    """
    Основной случай задачи Б: две строки длиной 300 (B получена из A правками).
    Возвращает расстояние, расшифровку и сопоставление реализаций.
    """
    rng = random.Random(SEED)
    a = random_text(rng, STRING_LEN)
    b = mutate(rng, a, MUTATIONS)
    D = edit_table(a, b)
    dist = D[len(a)][len(b)]
    ops = edit_script(a, b, D)
    assert apply_script(a, ops) == b, "восстановленная последовательность не даёт строку B"
    cost = sum(1 for o in ops if o[0] != "совпадение")
    assert cost == dist, "цена последовательности не равна расстоянию"
    assert edit_two_rows(a, b) == dist
    assert edit_memo(a, b)[0] == dist
    # контраст: две независимые случайные строки той же длины
    c = random_text(rng, STRING_LEN)
    dist_random = edit_two_rows(a, c)
    counts = {name: sum(1 for o in ops if o[0] == name)
              for name in ("совпадение", "замена", "удаление", "вставка")}
    return {"a": a, "b": b, "distance": dist, "ops": ops, "counts": counts,
            "distance_independent": dist_random, "cells": (len(a) + 1) * (len(b) + 1)}


def experiment_edit_naive():
    """
    Малые длины: наивный перебор, мемоизация и таблица. Берутся строки БЕЗ общих
    символов (A над первой половиной алфавита, B - над второй): это худший
    случай для перебора, число вызовов от строки к строке не «прыгает».
    Для каждой длины - число вызовов и время.
    """
    rng = random.Random(SEED + 1)
    half = len(ALPHABET) // 2
    rows = []
    for L in NAIVE_EDIT_LENGTHS:
        a = "".join(rng.choice(ALPHABET[:half]) for _ in range(L))
        b = "".join(rng.choice(ALPHABET[half:]) for _ in range(L))
        dn, cn = edit_naive(a, b)
        dm, cm = edit_memo(a, b)
        dt = edit_table(a, b)[L][L]
        assert dn == dm == dt
        rows.append({
            "L": L, "distance": dn, "calls_naive": cn, "calls_memo": cm, "cells": (L + 1) ** 2,
            "t_naive": avg_time(lambda: edit_naive(a, b), 0.1, 200),
            "t_memo": avg_time(lambda: edit_memo(a, b)),
            "t_table": avg_time(lambda: edit_table(a, b)),
        })
    return rows


def experiment_edit_scaling():
    """Рост времени таблицы и двухстрочной версии с длиной строк (ожидается Θ(L²))."""
    rng = random.Random(SEED + 2)
    rows = []
    for L in EDIT_LENGTHS:
        a, b = random_text(rng, L), random_text(rng, L)
        rows.append({
            "L": L, "cells": (L + 1) ** 2,
            "t_table": avg_time(lambda: edit_table(a, b), 0.2, 200),
            "t_two_rows": avg_time(lambda: edit_two_rows(a, b), 0.2, 200),
        })
    return rows


def experiment_cache():
```

```
    """
    Г6: собственная мемоизация (декоратор my_memoize), lru_cache и явный словарь
    (функции stairs_memo / edit_memo). Время вычисления с ПУСТЫМ кэшем и число
    обращений / реальных вычислений. Для задачи А - при n из варианта (27) и при
    больших n; для задачи Б - при длинах CACHE_EDIT_LENGTHS.
    """
    out = {"stairs": [], "edit": []}
    for n in [27] + CACHE_BIG_N:
        row = {"n": n}
        for kind in ("own", "lru"):
            def maker(kind=kind):
                g = make_stairs_decorated(kind)
                return (lambda: g(n)), g
            row["t_" + kind] = median_run(maker, 200 if n < 100 else 100)
            g = make_stairs_decorated(kind)
            g(n)
            row["calls_" + kind], row["misses_" + kind] = cache_calls(g, kind)
        row["t_dict"] = avg_time(lambda: stairs_memo(n))      # явный словарь внутри функции
        row["calls_dict"] = stairs_memo(n)[1]
        # проверка совпадения значений
        assert make_stairs_decorated("own")(n) == make_stairs_decorated("lru")(n) == stairs_memo(n)[0]
        out["stairs"].append(row)
    rng = random.Random(SEED + 3)
    for L in CACHE_EDIT_LENGTHS:
        a, b = random_text(rng, L), random_text(rng, L)
        row = {"L": L}
        for kind in ("own", "lru"):
            def maker(kind=kind):
                g = make_edit_decorated(kind, a, b)
                return (lambda: g(len(a), len(b))), g
            row["t_" + kind] = median_run(maker, 7)
            g = make_edit_decorated(kind, a, b)
            d = g(len(a), len(b))
            row["calls_" + kind], row["misses_" + kind] = cache_calls(g, kind)
            row["distance"] = d
        row["t_dict"] = avg_time(lambda: edit_memo(a, b), 0.2, 50)
        row["calls_dict"] = edit_memo(a, b)[1]
        row["t_table"] = avg_time(lambda: edit_table(a, b), 0.2, 50)
        assert row["distance"] == edit_table(a, b)[L][L]
        out["edit"].append(row)
    return out


# ---------------------------------------------------------------------------
# 6. САМОПРОВЕРКА И МАЛЫЕ ПРИМЕРЫ
# ---------------------------------------------------------------------------
def self_test():
    """
    Проверки: (1) три реализации задачи А совпадают с прямым перебором путей;
    (2) ДП для редакционного расстояния совпадает с полным перебором на 200
    парах малых строк; (3) восстановленная последовательность операций
    превращает А в В и стоит ровно D[m][n]. При ошибке - AssertionError.
    """
    # (1) прямое построение всех способов (последовательностей шагов 1, 2, 3)
    def brute_stairs(n):
        if n == 0:
            return 1
        return sum(brute_stairs(n - s) for s in (1, 2, 3) if n - s >= 0)
    for n in range(0, 16):
        assert stairs_naive(n)[0] == stairs_memo(n)[0] == stairs_iter(n)[0] == brute_stairs(n)
    # (2) и (3) редакционное расстояние
    rng = random.Random(0)
    for _ in range(200):
        la, lb = rng.randint(0, 7), rng.randint(0, 7)
        a = "".join(rng.choice("абв") for _ in range(la))      # малый алфавит - много совпадений
        b = "".join(rng.choice("абв") for _ in range(lb))
        D = edit_table(a, b)
        dist = D[la][lb]
        assert edit_naive(a, b)[0] == dist, "ДП не совпало с полным перебором"
        assert edit_memo(a, b)[0] == dist and edit_two_rows(a, b) == dist
        ops = edit_script(a, b, D)
        assert apply_script(a, ops) == b
        assert sum(1 for o in ops if o[0] != "совпадение") == dist
    print("Самопроверка пройдена (задача А: 3 реализации = перебор; задача Б: ДП = полный перебор на 200 парах)")


def demo_small():
    """Малые примеры: дерево рекурсии f(5), таблица ДП и расшифровка для двух слов."""
    tree, counts = recursion_tree_stairs(5)
    print("\n--- Дерево рекурсии наивной версии для f(5) (Mermaid) ---")
    print(tree)
    print("Число вхождений подзадач:", ", ".join(f"F({k}) x{v}" for k, v in sorted(counts.items(), reverse=True)),
          f"; всего вызовов {sum(counts.values())}")

    a, b = "СОБАКА", "ЛОПАТА"
```

```
    D = edit_table(a, b)
    print(f"\n--- Таблица ДП для слов «{a}» и «{b}» ---")
    print("      " + "   ".join(["·"] + list(b)))
    for i in range(len(a) + 1):
        label = "·" if i == 0 else a[i - 1]
        print(f"  {label}  " + "  ".join(f"{D[i][j]:>2}" for j in range(len(b) + 1)))
    ops = edit_script(a, b, D)
    print(f"Расстояние = {D[len(a)][len(b)]}. Расшифровка:")
    for name, i, j, ca, cb in ops:
        print(f"  {name:<11} {('A[' + str(i) + ']=' + ca) if ca else '':<9} {('B[' + str(j) + ']=' + cb) if cb else ''}")
    print("Результат применения:", apply_script(a, ops))


# ---------------------------------------------------------------------------
# 7. ГРАФИКИ
# ---------------------------------------------------------------------------
def make_plots(res):
    """Строит и сохраняет три рисунка (lr7_plot_stairs / _edit / _cache)."""
    rows = res["stairs"]
    ns = [r["n"] for r in rows]

    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
    ax = axes[0]
    ax.plot(ns, [r["calls_naive"] for r in rows], "ro-", label="наивная рекурсия")
    ax.plot(ns, [r["calls_memo"] for r in rows], "bs-", label="мемоизация")
    ax.plot(ns, [r["calls_iter"] for r in rows], "g^-", label="итеративная (шаги цикла)")
    base = res["fit"]["slope_base"]
    c0 = rows[0]["calls_naive"]
    ax.plot(ns, [c0 * base ** (n - ns[0]) for n in ns], "k--", lw=1, label=f"c^n, c = {base:.3f}")
    ax.set_yscale("log")
    ax.set_xlabel("n")
    ax.set_ylabel("число вызовов (лог. шкала)")
    ax.set_title("Задача А: число вызовов")
    ax.grid(True, which="both", alpha=0.3)
    ax.legend(fontsize=8)
    ax = axes[1]
    ax.plot(ns, [r["t_naive"] for r in rows], "ro-", label="наивная рекурсия")
    ax.plot(ns, [r["t_memo"] for r in rows], "bs-", label="мемоизация")
    ax.plot(ns, [r["t_iter"] for r in rows], "g^-", label="итеративная")
    ax.set_yscale("log")
    ax.set_xlabel("n")
    ax.set_ylabel("время, с (лог. шкала)")
    ax.set_title("Задача А: время работы")
    ax.grid(True, which="both", alpha=0.3)
    ax.legend(fontsize=8)
    fig.tight_layout()
    fig.savefig("lr7_plot_stairs.png", dpi=150)
    plt.close(fig)

    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
    ax = axes[0]
    nv = res["edit_naive"]
    ax.plot([r["L"] for r in nv], [r["calls_naive"] for r in nv], "ro-", label="полный перебор")
    ax.plot([r["L"] for r in nv], [r["calls_memo"] for r in nv], "bs-", label="мемоизация")
    ax.plot([r["L"] for r in nv], [r["cells"] for r in nv], "g^-", label="клеток таблицы (L+1)²")
    ax.set_yscale("log")
    ax.set_xlabel("длина строк L")
    ax.set_ylabel("число вызовов (лог. шкала)")
    ax.set_title("Задача Б: малые длины")
    ax.grid(True, which="both", alpha=0.3)
    ax.legend(fontsize=8)
    ax = axes[1]
    sc = res["edit_scaling"]
    ax.plot([r["L"] for r in sc], [r["t_table"] * 1e3 for r in sc], "bo-", label="таблица")
    ax.plot([r["L"] for r in sc], [r["t_two_rows"] * 1e3 for r in sc], "g^-", label="две строки")
    l0, t0 = sc[0]["L"], sc[0]["t_table"] * 1e3
    ax.plot([r["L"] for r in sc], [t0 * (r["L"] / l0) ** 2 for r in sc], "k--", lw=1, label="∝ L²")
    ax.set_xlabel("длина строк L")
    ax.set_ylabel("время, мс")
    ax.set_title("Задача Б: рост времени ДП")
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=8)
    fig.tight_layout()
    fig.savefig("lr7_plot_edit.png", dpi=150)
    plt.close(fig)

    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
    cs, ce = res["cache"]["stairs"], res["cache"]["edit"]
    for ax, data, key, title, unit in ((axes[0], cs, "n", "Задача А (лестница)", 1e6),
                                       (axes[1], ce, "L", "Задача Б (редакционное расстояние)", 1e3)):
        xs = range(len(data))
        w = 0.27
        ax.bar([x - w for x in xs], [d["t_own"] * unit for d in data], w, label="собственный декоратор")
        ax.bar(list(xs), [d["t_lru"] * unit for d in data], w, label="functools.lru_cache")
        ax.bar([x + w for x in xs], [d["t_dict"] * unit for d in data], w, label="явный словарь в функции")
```

```
        ax.set_xticks(list(xs))
        ax.set_xticklabels([f"{key} = {d[key]}" for d in data])
        ax.set_ylabel("время, мкс" if unit == 1e6 else "время, мс")
        ax.set_yscale("log")
        ax.set_title(title)
        ax.grid(True, axis="y", which="both", alpha=0.3)
        ax.legend(fontsize=8)
    fig.tight_layout()
    fig.savefig("lr7_plot_cache.png", dpi=150)
    plt.close(fig)


# ---------------------------------------------------------------------------
# 8. ВЫВОД РЕЗУЛЬТАТОВ
# ---------------------------------------------------------------------------
def system_info():
    """Строка с характеристиками системы (вставляется в отчёт)."""
    cpu = platform.processor() or platform.machine()
    try:                                          # на Linux модель процессора лежит в /proc/cpuinfo
        with open("/proc/cpuinfo", encoding="utf-8") as f:
            for line in f:
                if line.startswith("model name"):
                    cpu = line.split(":", 1)[1].strip()
                    break
    except OSError:
        pass
    return (f"ОС: {platform.system()} {platform.release()}; процессор: {cpu}; "
            f"Python {platform.python_version()}")


def print_report(res):
    """Печатает таблицы результатов в консоль."""
    print("\nТаблица 1. Задача А: значение f(n), число вызовов и время")
    print(f"  {'n':>3}{'f(n)':>12}{'вызовов (наив.)':>17}{'вызовов (мемо)':>16}{'шагов (итер.)':>15}"
          f"{'t наив., с':>13}{'t мемо, мкс':>13}{'t итер., мкс':>14}")
    for r in res["stairs"]:
        print(f"  {r['n']:>3}{r['value']:>12}{r['calls_naive']:>17}{r['calls_memo']:>16}{r['calls_iter']:>15}"
              f"{r['t_naive']:>13.5f}{r['t_memo'] * 1e6:>13.2f}{r['t_iter'] * 1e6:>14.2f}")
    f = res["fit"]
    print(f"  Основание экспоненты: T(n+1)/T(n) при n = 27: {f['last_ratio']:.4f}; среднее: "
          f"{f['mean_ratio']:.4f}; по наклону log T: {f['slope_base']:.4f}; теория (корень x³ = x² + x + 1): 1.8393")

    print("\nТаблица 2. Предел наивной рекурсии (порог 10 с)")
    for r in res["limit"]:
        print(f"  n = {r['n']}: {r['t']:.2f} с, {r['calls']} вызовов")

    e = res["edit_main"]
    print(f"\nТаблица 3. Задача Б: строки длиной {STRING_LEN}, A -> B ({MUTATIONS} случайных правок)")
    print(f"  Редакционное расстояние = {e['distance']} (для двух независимых случайных строк: {e['distance_independent']})")
    print("  Состав: " + ", ".join(f"{k} - {v}" for k, v in e["counts"].items()))
    print("  Расшифровка (без совпадений): №, операция, позиция в A, позиция в B, символы")
    k = 0
    for name, i, j, ca, cb in e["ops"]:
        if name != "совпадение":
            k += 1
            print(f"    {k:>3}. {name:<8} A[{i}]={ca or '-'}  B[{j}]={cb or '-'}")

    print("\nТаблица 4. Задача Б, малые длины: число вызовов и время")
    print(f"  {'L':>3}{'D':>4}{'вызовов (перебор)':>19}{'вызовов (мемо)':>16}{'клеток':>8}"
          f"{'t перебор, мкс':>16}{'t мемо, мкс':>13}{'t таблица, мкс':>16}")
    for r in res["edit_naive"]:
        print(f"  {r['L']:>3}{r['distance']:>4}{r['calls_naive']:>19}{r['calls_memo']:>16}{r['cells']:>8}"
              f"{r['t_naive'] * 1e6:>16.1f}{r['t_memo'] * 1e6:>13.1f}{r['t_table'] * 1e6:>16.1f}")

    print("\nТаблица 5. Рост времени ДП с длиной строк L (мс)")
    print(f"  {'L':>4}{'клеток':>9}{'таблица':>10}{'две строки':>12}")
    for r in res["edit_scaling"]:
        print(f"  {r['L']:>4}{r['cells']:>9}{r['t_table'] * 1e3:>10.2f}{r['t_two_rows'] * 1e3:>12.2f}")

    print("\nТаблица 6 (Г6). Мемоизация: собственный декоратор / lru_cache / явный словарь")
    print("  Задача А (время в мкс, пустой кэш)")
    print(f"  {'n':>5}{'own':>10}{'lru':>10}{'словарь':>10}{'обращений own':>15}{'вычислений':>12}")
    for r in res["cache"]["stairs"]:
        print(f"  {r['n']:>5}{r['t_own'] * 1e6:>10.2f}{r['t_lru'] * 1e6:>10.2f}{r['t_dict'] * 1e6:>10.2f}"
              f"{r['calls_own']:>15}{r['misses_own']:>12}")
    print("  Задача Б (время в мс, пустой кэш)")
    print(f"  {'L':>5}{'own':>10}{'lru':>10}{'словарь':>10}{'таблица':>10}{'обращений':>11}{'вычислений':>12}")
    for r in res["cache"]["edit"]:
        print(f"  {r['L']:>5}{r['t_own'] * 1e3:>10.2f}{r['t_lru'] * 1e3:>10.2f}{r['t_dict'] * 1e3:>10.2f}"
              f"{r['t_table'] * 1e3:>10.2f}{r['calls_own']:>11}{r['misses_own']:>12}")


def main():
    print("Лабораторная работа № 7, вариант 13")
    print(system_info())
```

```
    print(f"Задача А: лестница с шагами 1, 2, 3, n = {STAIRS_N[0]}...{STAIRS_N[-1]}; "
          f"задача Б: редакционное расстояние, строки длиной {STRING_LEN}")
    self_test()
    demo_small()

    res = {}
    print("\nЗадача А ...", flush=True)
    res["stairs"] = experiment_stairs()
    res["fit"] = fit_base(res["stairs"])
    print("  поиск предела наивной рекурсии:", flush=True)
    res["limit"] = experiment_limit(res["stairs"][-1]["n"], res["stairs"][-1]["t_naive"])
    print("Задача Б ...", flush=True)
    res["edit_main"] = experiment_edit_main()
    res["edit_naive"] = experiment_edit_naive()
    res["edit_scaling"] = experiment_edit_scaling()
    print("Г6: сравнение мемоизаций ...", flush=True)
    res["cache"] = experiment_cache()
    print_report(res)
    make_plots(res)
    print("\nГрафики сохранены: lr7_plot_stairs.png, lr7_plot_edit.png, lr7_plot_cache.png")
    if "--json" in sys.argv:                          # служебный вывод для отчёта
        path = sys.argv[sys.argv.index("--json") + 1]
        with open(path, "w", encoding="utf-8") as fh:
            json.dump(res, fh, ensure_ascii=False, indent=1)
    if "--show" in sys.argv:
        plt.show()


if __name__ == "__main__":
    main()
```

5. Характеристики вычислительной системы

Параметр	Значение
Процессор	Intel(R) Xeon(R) Processor @ 2.10GHz, 2 логических ядра
Оперативная память	8 ГБ
Операционная система	Linux (Ubuntu 24.04)
Интерпретатор	Python 3.13.18
Таймер	time.perf_counter()

Абсолютные значения времени зависят от системы; числа вызовов, значения f(n) и расстояние не зависят от неё вообще, а форма роста времени и соотношения между реализациями должны сохраниться. Программа при запуске выводит характеристики системы автоматически.

6. Результаты измерений

6.1. Задача A: лестница с шагами 1, 2, 3

Таблица 1. Число вызовов и время трёх реализаций

n	f(n)	Вызовов, наивная	Вызовов, мемоизация (3n − 5)	Шагов цикла, итеративная	Время, наивная, мс	Время, мемоизация, мкс	Время, итеративная, мкс
10	274	289	25	9	0,022	3,28	0,44
11	504	532	28	10	0,040	3,65	0,48
12	927	979	31	11	0,076	3,97	0,56
13	1 705	1 801	34	12	0,145	4,44	0,58
14	3 136	3 313	37	13	0,259	4,69	0,63
15	5 768	6 094	40	14	0,486	5,86	0,72
16	10 609	11 209	43	15	0,889	5,59	0,74
17	19 513	20 617	46	16	1,716	5,76	1,42
18	35 890	37 921	49	17	2,946	6,13	0,87
19	66 012	69 748	52	18	5,424	6,39	0,93
20	121 415	128 287	55	19	10,667	7,08	0,99
21	223 317	235 957	58	20	18,859	7,19	1,02
22	410 744	433 993	61	21	33,944	7,57	1,06
23	755 476	798 238	64	22	63,974	7,91	1,15
24	1 389 537	1 468 189	67	23	118,721	8,39	1,20
25	2 555 757	2 700 421	70	24	219,485	8,49	1,22
26	4 700 770	4 966 849	73	25	447,027	10,01	1,28
27	8 646 064	9 135 460	76	26	778,871	9,39	1,67

Число вызовов наивной версии полностью совпадает с формулой $T(n) = 1 + T(n − 1) + T(n − 2) + T(n − 3)$ (проверено программой для каждого n), а мемоизированной — с $3n − 5$. Отношение $T(n + 1)/T(n)$ при $n = 27$ равно 1,8393, среднее по всему диапазону — 1,8395, оценка по наклону прямой $\log T(n)$ — 1,8394; теоретическое значение постоянной трибоначчи 1,8393.

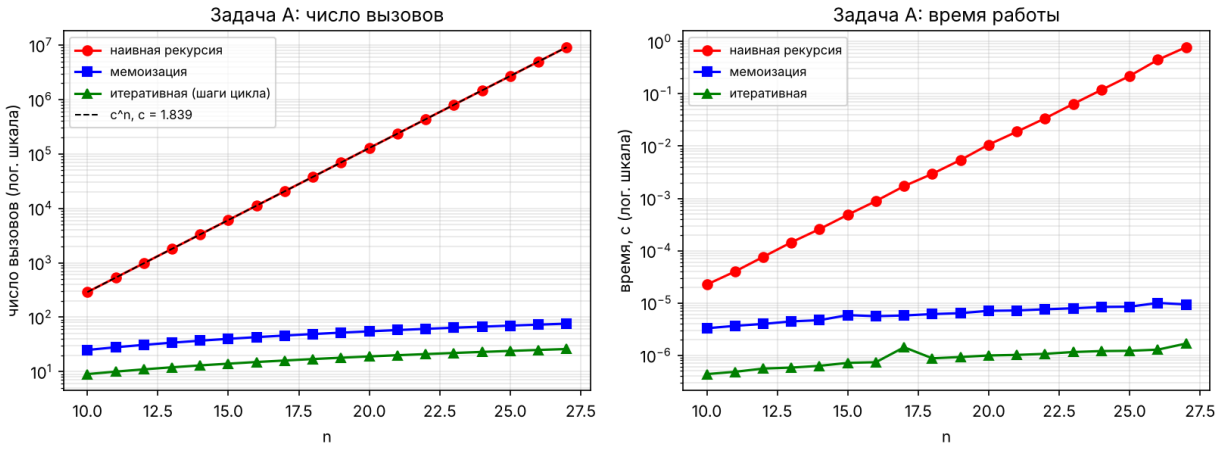


Рис. 6. Задача А: число вызовов и время работы (логарифмические шкалы). Штриховая линия — c^n, $c = 1{,}839$

Таблица 2. Предел наивной рекурсии (порог 10 с)

n	Вызовов	Время, с	Результат
28	16 802 731	1,35	в пределах 10 с
29	30 905 041	2,60	в пределах 10 с
30	56 843 233	4,65	в пределах 10 с
31	104 551 006	8,39	в пределах 10 с
32	192 299 281	15,85	**больше 10 с**

Наивная рекурсия перестаёт завершаться за 10 секунд при n = 32 (15,8 с); при n = 31 она ещё укладывается (8,4 с). Для сравнения, мемоизация и итеративная версия при n = 32 занимают микросекунды.

6.2. Задача Б: редакционное расстояние

Малый пример. Слова «СОБАКА» → «ЛОПАТА»: расстояние 3 (замены С→Л, Б→П, К→Т); таблица ДП:

	·	Л	О	П	А	Т	А
·	0	1	2	3	4	5	6
С	1	1	2	3	4	5	6
О	2	2	1	2	3	4	5
Б	3	3	2	2	3	4	5
А	4	4	3	3	2	3	4
К	5	5	4	4	3	3	4
А	6	6	5	5	4	4	3

Таблица 3. Малые длины: полный перебор, мемоизация, таблица (строки без общих символов — худший случай для перебора)

L	D	Вызовов, перебор	Вызовов, мемоизация	Клеток таблицы (L+1)²	Время, перебор, мкс	Время, мемоизация, мкс	Время, таблица, мкс
2	2	19	13	9	3,2	3,4	1,4
3	3	94	28	16	11,6	6,9	2,4
4	4	481	49	25	58,7	11,8	3,5
5	5	2 524	76	36	313,2	19,1	4,7
6	6	13 483	109	49	1636,8	25,3	6,4
7	7	72 958	148	64	8858,3	34,6	8,2
8	8	398 593	193	81	49027,0	44,8	10,3

Число вызовов перебора растёт в ≈ 5,5 раза при увеличении длины на 1 (теория ≈ 5,83), при L = 8 это уже 398 593 вызовов против 81 клеток таблицы. Для L = 300 перебор невозможен: порядка $5{,}83^{300}$ вызовов.

Таблица 4. Рост времени ДП с длиной строк (случайные строки, ожидается Θ(L²))

L	Клеток	Таблица, мс	Две строки, мс	Время / клетка, нс
50	2 601	0,34	0,30	131
100	10 201	1,19	1,25	117
150	22 801	2,70	2,75	118
200	40 401	4,86	5,04	120
300	90 601	11,70	11,97	129
400	160 801	23,71	21,44	147

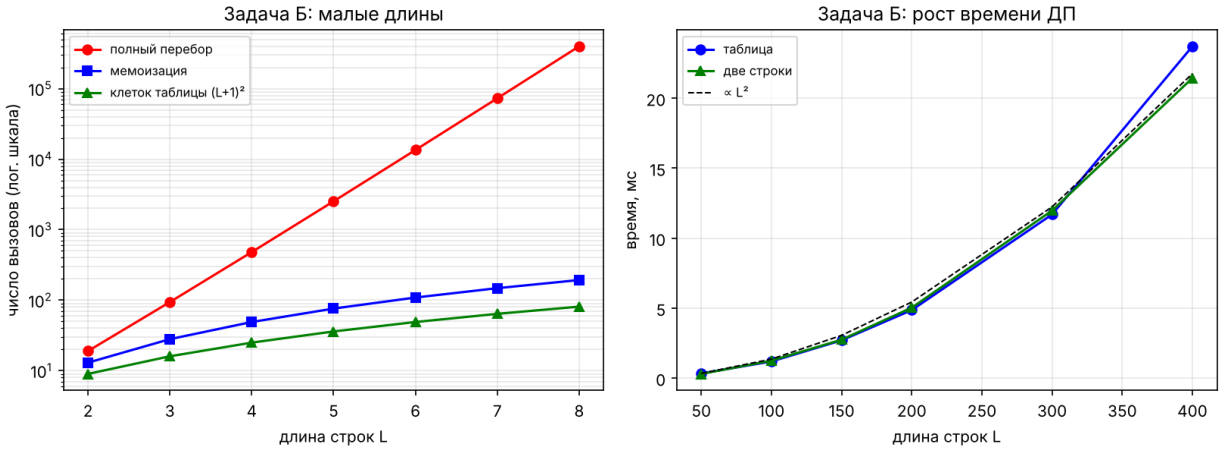


Рис. 7. Задача Б: число вызовов на малых длинах (слева) и рост времени ДП с длиной строк (справа, штриховая линия — ∝ L²)

6.3. Дополнительное задание Г6: собственная мемоизация и `lru_cache`

Сравниваются три способа мемоизации одной и той же рекурсии: **собственный декоратор** `my_memoize` (обёртка на чистом Python со словарём), **functools.lru_cache(maxsize=None)** и **явный словарь внутри функции** (`stairs_memo`, `edit_memo`). Время — медиана вычисления с *пустым* кэшем.

Таблица 5. Задача А: время вычисления f(n) с пустым кэшем, мкс

n	Собственный декоратор	lru_cache	Явный словарь	Обращений к функции	Реальных вычислений	own / lru
27	13,00	5,80	9,34	76	28	2,24
100	50,74	21,48	35,60	295	101	2,36
300	200,48	76,29	131,24	895	301	2,63
900	716,57	324,12	516,69	2695	901	2,21

Таблица 6. Задача Б: время вычисления редакционного расстояния со случайными строками, мс

L	Собственный декоратор	lru_cache	Явный словарь	Таблица (ДП)	Обращений	Вычислений	own / lru
100	8,02	4,26	6,86	1,16	28 590	9 939	1,88
200	35,93	27,91	32,50	4,72	112 887	38 969	1,29
300	89,41	53,80	73,92	11,02	253 898	87 281	1,66

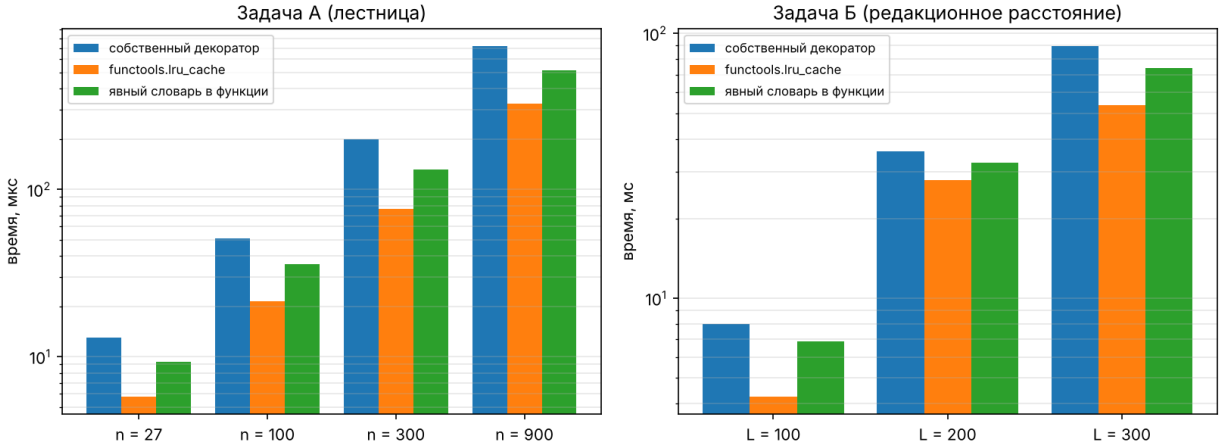


Рис. 8. Г6: время мемоизированных реализаций (логарифмическая шкала)

Объяснение разницы. Число вычислений у всех мемоизаций одинаково (n + 1 для задачи А; 87 281 различных пар (i, j), достижимых из (300, 300), для задачи Б): мемоизация убирает повторные вычисления в любой форме. Различается стоимость *одного обращения* к кэшу. У `lru_cache` обёртка написана на языке C: формирование ключа из аргументов, поиск в словаре и возврат закэшированного значения выполняются без создания кадра Python-функции. Собственный декоратор — это обычная функция Python: при каждом обращении создаётся кадр обёртки, упаковываются аргументы `*args`, обновляются счётчики, выполняются проверка `in` и повторное чтение словаря, а при промахе добавляется ещё вызов исходной функции — на каждый уровень рекурсии приходится два кадра Python. Поэтому `lru_cache` быстрее собственного декоратора в 1,3–2,6 раза. Явный словарь внутри функции быстрее декоратора (нет лишнего кадра и счётчиков), но медленнее `lru_cache`; а восходящая таблица без рекурсии быстрее лучшей мемоизации примерно в 4,9 раза при L = 300.

7. Оптимальное решение задачи Б (с расшифровкой)

Строки длиной 300 символов; B получена из A тридцатью случайными правками.

Строка A:

```
икццюехзцдзхшедвтжшкаодяцфаиядвююююияпшодюимзрэтуоцмохлхгмяфюхичптфшдпцшптекжесмимпифиндурт
```

зузжчмдвоххпнашобздрючяпхиешошиэюмжчхмсчцдчфзшржфбучубмушулекжеюцтгзевохфтютчбпялхвсадкгок
иуфчврсмябаллдрттсярзцжкюбнсчешдиоцяжиоузмэзукожхмоблнпввэбфахэкжпртнлбксоддбгмяежккфнкбмд
гмрпууобмткдюблетшдазцдврмвмию

Строка B:

икццюехгзцдрхтшедвжшгкаодяфамдвюююияпшодюримзрэлтуоцмаохлхгмяфюичпфшдпцшпцтекифсмимфпифину
ртзузжчамдвохухпнашочбздрючяпхиешошиэцмжчхмсичцдчфзшржбучубмушлекжтаютгзевохфтютчбпялхвсад
кгокиуфчврсмзаллдрттснрзцжкюбнсчешдиоцяэжузмэзукжхмобнпввджбфахэжпрятялбксоддгмяежккфнкюбм
дгмрнуобмтбдюблетшдазцдврмвмию

Редакционное расстояние D[300][300] = 46. Оптимальная последовательность состоит из 46 операций: замен — 16, удалений — 15, вставок — 15; ещё 269 символов остаются без изменений (всего 315 шагов пути по таблице). Для двух независимых случайных строк той же длины расстояние составило бы 270. Применение полученной последовательности к строке A даёт в точности строку B (проверено программой), а её цена равна D[300][300]. Позиции указаны в исходных строках (с 1); при вставке указана позиция в A, *после* которой вставляется символ.

Таблица 7. Расшифровка оптимального решения (операции без учёта совпадений)

№	Операция	Поз. A	Симв. A	Поз. B	Симв. B
1	вставка	после 7	—	8	г
2	замена	11	з	12	р
3	вставка	после 12	—	14	т
4	удаление	17	т	—	—
5	вставка	после 19	—	21	г
6	удаление	25	ц	—	—
7	удаление	28	и	—	—
8	замена	29	я	29	м
9	вставка	после 41	—	42	р
10	вставка	после 46	—	48	л
11	вставка	после 51	—	54	а
12	удаление	61	х	—	—
13	удаление	65	т	—	—
14	вставка	после 72	—	74	ц
15	замена	76	ж	78	и
16	замена	77	е	79	ф
17	вставка	после 81	—	84	ф
18	удаление	87	д	—	—
19	вставка	после 95	—	98	а
20	вставка	после 100	—	104	у
21	вставка	после 106	—	111	ч
22	замена	123	ю	128	ц
23	вставка	после 129	—	135	и
24	удаление	139	ф	—	—
25	удаление	148	у	—	—
26	замена	153	е	157	т
27	замена	154	ю	158	а
28	замена	155	ц	159	ю
29	удаление	189	я	—	—
30	замена	190	б	193	з
31	замена	199	я	202	н
32	удаление	217	ж	—	—
33	замена	218	и	220	э
34	замена	219	о	221	ж
35	удаление	227	о	—	—
36	удаление	233	л	—	—
37	вставка	после 237	—	238	д
38	замена	238	э	239	ж
39	удаление	244	к	—	—
40	вставка	после 247	—	248	я
41	замена	249	н	250	я
42	удаление	257	б	—	—
43	вставка	после 267	—	268	ю
44	удаление	274	п	—	—
45	замена	275	у	275	н
46	замена	281	к	281	б

8. Выводы

1. **Наивная рекурсия экспоненциальна.** Число вызовов растёт в 1,839 раза при увеличении n на 1 (теория — корень уравнения $x^3 = x^2 + x + 1$, $c \approx 1{,}8393$): при n = 27 это 9 135 460 вызовов и 0,78 с, при n = 10 — 289 вызовов. Предел 10 с достигается при n = 32 (15,8 с; при n = 31 — 8,4 с).
2. **Причина — перекрывающиеся подзадачи.** В дереве рекурсии для f(5) (рис. 1) из 13 вызовов лишь 6 различных подзадач: F(2) вычисляется 4 раза, F(1) — 3 раза. Мемоизация вычисляет каждую подзадачу один раз и сокращает число вызовов до $3n - 5$ (76 при n = 27 вместо 9 135 460): сложность падает с $\Theta(c^n)$ до $\Theta(n)$, платой

служит Θ(n) памяти под кэш и глубина рекурсии n.

3. **Итеративное решение лучше мемоизации.** Оно использует O(1) памяти, не ограничено глубиной стека и быстрее: при n = 27 — 1,67 мкс против 9,39 мкс у мемоизации (5,6 раза) и 779 мс у наивной версии (в 465 325 раз медленнее итеративной).
4. **ДП для редакционного расстояния.** Таблица (m + 1)(n + 1) = 90 601 клеток для строк длиной 300 заполняется за 12 мс; время растёт квадратично (при удвоении L с 100 до 200 и с 150 до 300 — в 4,1 и 4,3 раза при теоретических 4), а на клетку приходится почти постоянное время (131–147 нс). Полный перебор на тех же данных невозможен: уже при L = 8 он делает 398 593 вызовов (против 81 клеток), рост ≈ 5,2–5,5 раза на каждый символ (теория → 5,83).
5. **Корректность.** ДП совпало с полным перебором на 200 парах строк, а восстановленная последовательность операций (46 операций для строк из раздела 7) превращает A в B и стоит ровно D[300][300]. Расстояние 46 заметно меньше, чем у независимых случайных строк (270), как и ожидается для родственных строк. Двухстрочная версия даёт то же расстояние при O(n) памяти, но не позволяет восстановить операции (время почти такое же — 12,0 против 11,7 мс).
6. **Г6: lru_cache против собственной мемоизации.** Все варианты делают одинаковое число вычислений, но `lru_cache` (реализован на C) быстрее собственного декоратора в 1,3–2,6 раза: каждое обращение к собственному декоратору порождает лишний кадр Python-функции, упаковку аргументов и счётчики. Явный словарь внутри функции — посередине. Однако даже лучшая мемоизация проигрывает восходящей таблице (при L = 300: 53,80 мс против 11,02 мс, в 4,9 раза), а рекурсивная мемоизация требует увеличения предела рекурсии (в программе — 20 000).
7. **Практический смысл.** Редакционное расстояние используется для проверки правописания, поиска похожих названий и адресов контрагентов, сравнения версий текста и последовательностей (дубли в справочнике клиентов). ДП делает такую задачу точно разрешимой за время порядка mn, а восстановление операций показывает, *какие* именно правки превращают одну строку в другую.

9. Список литературы

1. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013. — Гл. 15.
2. Рафгарден Т. Совершенный алгоритм. Жадные алгоритмы и динамическое программирование. — СПб.: Питер, 2020.
3. Скиена С. Алгоритмы. Руководство по разработке. — 3-е изд. — СПб.: БХВ-Петербург, 2022. — Гл. 10.
4. Левитин А. Алгоритмы. Введение в разработку и анализ. — М.: Вильямс, 2006.
5. Документация Python: модуль functools, декоратор lru_cache [Электронный ресурс]. — URL: https://docs.python.org/3/library/functools.html